Reversed: Voiceflow as backbone, replaced by a custom Node graph

Eight engineering days to leave Voiceflow. The uncertainty gate is a derived value across four signals with a per-intent override; a block graph could carry one of them.

28 MAY 2026 · 7 MIN READ

The first Lina prototypes ran on Voiceflow. It was the right call for client demos. It was the wrong call for the uncertainty gate, which is the thing that actually keeps the agent from booking a chest-pain caller for next Tuesday. Here is the point at which the platform stopped fitting and what runs now.

What shipped first

Voiceflow as the backbone for the first three pilot bots. Intent blocks for routing, knowledge base blocks for clinic-specific FAQs, function blocks for external calls into the Lina backend, and the live transcript piped into the Voiceflow runtime via a custom carrier. The visual editor was a real win in client conversations: when the practice owner could see the flow, the onboarding briefing took twenty minutes instead of two hours. Three demos closed off the back of a screen share with the canvas open.

For the first six weeks of pilot one this was a fine choice. It would probably still be a fine choice for a clinic that wants a chatbot to answer opening hours and clinic address questions.

The signal that forced the reversal

The Lina safeguard contract has four things in it: human handoff always one click away, uncertainty gate at 0.75 confidence, no-gaslighting (the agent never offers help it cannot deliver), transparency (caller knows it is an agent, full conversation logged). The first three of these became hard to express in Voiceflow blocks at roughly the same time.

The uncertainty gate was the one that ended it. The 0.75 threshold is not a single number applied to a single signal; it is a derived value across ASR confidence, LLM logprob on the chosen response, semantic similarity of the response to the question, and a per-intent override (medical-triage utterances escalate at 0.92, not 0.75). Voiceflow exposed intent confidence and that was about it. To get the LLM logprob into a block-graph decision the work had to happen in a function block, then thread the value through session variables, then get checked again in a separate function block before the response was committed. The graph started to look like spaghetti within two pilots.

The no-gaslighting safeguard surfaced the second problem. The agent has to refuse to confirm something it does not actually know. In a block model that meant scattering "did the agent just confirm something" checks across every response-emitting block, with the side-channel of a global "did the user ask a yes/no question" flag. Adding a new response path meant remembering to add both checks. Forgetting either was easy. One got missed on a Wednesday and hotfixed on a Thursday.

What runs now

A custom Node graph in TypeScript. Each node has first-class access to the full conversational state (ASR confidence, LLM logprobs, intent override table, the history of every prior turn). The four safeguards are cross-cutting middlewares that wrap every node, not blocks the graph author has to remember to wire in. The uncertainty gate is a single function with explicit unit tests; if it does not pass on a given utterance, no node downstream runs, and the call escalates.

The graph editor is gone. Briefings are now read documents (not visual canvas walks), which is also why the briefing template lives at /method instead of being a Voiceflow screen-share. The demo polish is gone. What came back is the ability to make a safety argument with the code in hand.

What the migration actually cost

Eight engineering days end to end. Three days to write the graph runtime (it is small: about 700 lines plus tests). Two days to port the existing three pilots. Two days to write the middleware suite for the four safeguards with

unit tests. One day to write the new briefing template that replaced the Voiceflow screen-share at the start of every engagement.

Voiceflow itself was about 80 EUR a month at the tier in use. The replacement runs on the same Node process that owns the rest of the call graph, no incremental hosting cost.

Where Voiceflow is still the right answer

If your safety surface is "make sure the bot does not promise a refund it cannot give", Voiceflow handles that fine. If your buyer is a marketing lead who wants to be able to read the flow without asking an engineer, Voiceflow is the right answer. If your client wants to edit the flow between engagements without paying you to do it, Voiceflow is the right answer.

It stops being the right answer when the safety constraint is expressed in terms the block model cannot carry. For medical voice agents in Germany that line gets crossed inside the first month.

The lesson

Low-code is great until you need to express something the low-code platform cannot model. The mistake was not picking Voiceflow. The mistake was believing that the demos closing meant the platform was scaling with the product; what was actually happening was that the demo and the production bot had started to diverge, and the production bot was quietly being patched in code on the side. When a tool is silently being replaced by a shadow tool, the tool has lost.

Fabi writes about backend systems, data pipelines and the protocols underneath them. More at **github.com/Fabi-SPL/findings**.
Originally published at https://lucidailabs.com/essays-reversed-voiceflow.
© 2026 Fabian Ilg. Licensed CC BY-NC-ND 4.0. Share it with attribution, do not sell it, do not publish altered versions.